

```
+ } else if (x == 0) {
+   "Zero"
+ } else {
+   "Greater than zero"
+ }
[1] "Greater than zero"
```

The *repeat* expression can be used to produce a looping construct in R. The syntax for 'repeat' is as follows:

`repeat expression`

For example:

```
> i <- 1

> repeat { if (i > 5) break else { print(i); i <- i + 1 } }
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

You will need to use the *break* keyword to exit from the loop. The *while* statement syntax is as follows:

`while (condition) expression`

Here's an example:

```
> i <- 1

> while (i <= 5) { print(i); i <- i + 1 }
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

R also has the *for* loop for performing iterations. Its syntax is as follows:

`for (var in list) expression`

An example is given below:

```
> for (i in seq(from=1, to=5, by=1)) print(i)
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

You can exit from the R interactive console session using *q()*.

R coding style

A number of coding styles exist for the R programming language such as Google's R style guide, Tidyverse style guide, etc. We will look at the recommended practices from the Tidyverse style guide. You can use *styler* and *lintr* R tools to convert and check that your R code conforms to the style guide.

1. All R source files must have a file name extension *.R*. For example, *critical.values.R*.
2. The variables and function names should use snake case style, such as *str_length*, *as_bare_character* and *boundary*.
3. It is a good practice to use a space after a comma for readability. For example:

```
> log(10, 2)
[1] 3.321928
```

4. Avoid a space with parenthesis (inside and outside) for a function call:

```
> proc(90, sin)
```

5. Use a space around infix operators like '+', '-', '==', '<-', etc.

```
> 1 + 2
[1] 3
```

There are exceptions when used with operators having a high precedence.

6. You can align '=' and '<-' expressions for better readability, as illustrated below:

```
inside <- 0
failure <- 0
```

7. If you wish to override the default value for a function argument, mention the full name:

```
stri_opts_regex(case_insensitive = ignore_case, ...)
```

8. Use two spaces for indentation. For example:

```
opts <- function(x) {
  if (identical(x, "")) {
    stri_opts_brkiter(type = "character")
  } else {
    attr(x, "options")
  }
}
```